Project 4: Implementation Guide (Part 1 + Part 2)

Traffic Accident Severity Analytics and Prediction | what to build, in what order, and how to know it's done

1. What you're building

One system that grows in two stages. Part 1 is the data engineering foundation; Part 2 sits on top of it and turns it into an MLOps solution.

```
UK accident CSVs + Open-Meteo API (+ optional OSM)
  -> raw landing zone
  -> Airflow ETL (extract, clean, validate, join)
  -> staging -> cleaned -> PostgreSQL/PostGIS
  -> accident mart + model-ready table
  -> Streamlit dashboard
  -> (Part 2) feature pipeline -> training + MLflow -> model registry
  -> FastAPI in Docker -> Streamlit prediction page
  -> monitoring (drift, latency, false negatives) -> retraining criteria
```

Build Part 1 so that Part 2 plugs into it without rework. In particular, keep a stable **model-ready table** from the start.

2. Tech stack and repo layout

Need	Tool
Ingestion	Python, Pandas, `requests`
Orchestration	Apache Airflow (run via Docker Compose)
Storage	PostgreSQL (+ PostGIS if you want spatial queries)
Dashboard	Streamlit (recommended, since Part 2 needs a prediction form)
Modelling	scikit-learn + Random Forest, XGBoost or LightGBM

Need	Tool
Experiment tracking and registry	MLflow
Serving	FastAPI + Docker
Config and secrets	`.env` file (never committed)

Suggested repo structure:

```
project4/
  docker-compose.yml        # postgres, airflow, mlflow, api, streamlit
  .env.example              # placeholders only, no real secrets
  README.md
  data/
    raw/                    # untouched downloads, API responses
    rejected/               # rejected-record files
  dags/
    accident_pipeline_dag.py
  src/
    ingestion/              # download_stats19.py, fetch_weather.py
    etl/                    # clean.py, validate.py, join_weather.py,
build_mart.py
    db/                     # schema.sql, load.py
    ml/                     # features.py, train.py, evaluate.py
    monitoring/             # drift.py, performance.py
  api/                      # FastAPI app + Dockerfile
  dashboard/                # Streamlit app
  docs/                     # architecture diagram, data dictionary, report
  tests/
```

3. Build order

No dates, just dependencies. Don't start a step until the one above it works.

1. Download one year of data, explore it in a notebook, and understand every column.
2. Start Postgres in Docker, define the schemas.
3. Write ingestion scripts and the raw layer, with logging.
4. Write cleaning and validation, producing the rejected-record log.
5. Fetch weather and join it to accidents.
6. Build dimensions, the fact table, aggregates, and the model-ready table.

7. Wrap steps 3 to 6 in an Airflow DAG and get one clean end-to-end run.
8. Build the Streamlit dashboard (5+ views).
9. Write the Part 1 documentation: diagram, data dictionary, README, report, screenshots.
10. Extend to more years of data if you haven't already (needed for a chronological split).
11. Features and training pipeline, then MLflow tracking.
12. Register the model, build the FastAPI service, containerize it.
13. Add the prediction page to Streamlit.
14. Monitoring and retraining design.
15. Final documentation, demo run-through, and submission package.

PART 1: Data Pipeline

4. Data acquisition

Primary dataset: UK Road Safety Open Data (DfT STATS19), found on data.gov.uk. It ships as three CSVs per year:

- **Collisions** (accident-level: date, time, lat/long, road type, speed limit, weather, light, road surface, severity, number of vehicles and casualties)
- **Vehicles** (one row per vehicle: type, manoeuvre, driver age and sex)
- **Casualties** (one row per casualty: age, sex, class, severity)

They link through the collision/accident index column. Column names differ slightly between years (`accident_index` vs `collision_index`), so check the actual headers in the file you download.

Severity coding: 1 = Fatal, 2 = Serious, 3 = Slight. Map these to readable labels during cleaning.

Weather source: Open-Meteo historical API. Free, no key needed for non-commercial use.

Optional: OpenStreetMap road context. Skip unless everything else is done.

For the report, record: source name, URL, licence (UK Open Government Licence for DfT data), the date you downloaded it, and step-by-step access instructions.

5. Ingestion

Write scripts that are **repeatable** (running twice gives the same result, no duplicates) and **logged**.

- `download_stats19.py`: downloads the CSVs to `data/raw/<source>/<extraction_date>/`. Never modify these files.
- `fetch_weather.py`: calls Open-Meteo and saves each JSON response untouched to `data/raw/weather/`.
- Keep an **ingestion log table** (`ops.ingestion_log`) with: run id, source name, extraction timestamp, file or API status (success / failed), row count, error message.
- Wrap network calls with timeouts and retries (for example exponential backoff). On failure, log it and continue or fail the task cleanly; never crash silently.
- Read big CSVs with explicit dtypes, or in chunks, to keep memory in check.

6. Data layers and schema

Use separate Postgres schemas for each layer:

Schema	Contents
`raw`	Files on disk, plus optionally raw tables loaded as text
`staging`	Tables mirroring the CSVs, minimal typing changes
`clean`	Deduplicated, validated, standardized records
`mart`	Star schema, aggregates, and the model-ready table
`ops`	Ingestion log, rejected records, data-quality results

Star schema for the mart:

- **fact_accident**: one row per accident, with keys to every dimension plus measures (number of vehicles, number of casualties, severity key, weather measures).
- **dim_date**: date, year, month, day, weekday, hour, is_weekend, is_rush_hour.
- **dim_road**: road type, speed limit, junction detail, road surface condition.

- **dim_weather**: weather condition category, light condition, plus binned temperature, rain and wind values.
- **dim_location**: police force, local authority, lat/long, grid cell id, urban or rural.
- **dim_severity**: code, label (Fatal / Serious / Slight).

Also load **dim_vehicle** and **dim_casualty** (or fact tables for them) so you can do the vehicle and casualty analysis view.

Example DDL pattern (adapt names to your columns):

```
CREATE SCHEMA IF NOT EXISTS clean;
CREATE TABLE IF NOT EXISTS clean.accident (
  accident_id      TEXT PRIMARY KEY,
  accident_ts      TIMESTAMP NOT NULL,
  latitude         DOUBLE PRECISION NOT NULL,
  longitude        DOUBLE PRECISION NOT NULL,
  severity_code    SMALLINT NOT NULL CHECK (severity_code IN (1,2,3)),
  road_type        TEXT,
  speed_limit      SMALLINT,
  weather_code     SMALLINT,
  light_code       SMALLINT,
  road_surface     SMALLINT,
  num_vehicles     SMALLINT,
  num_casualties   SMALLINT
);
```

7. Cleaning and validation rules

Implement each rule as a function that returns passing rows and rejected rows with a reason. Write every rejected row to `ops.rejected_records` (and/or a CSV in `data/rejected/`) with: source table, record id, rule name, reason, timestamp.

Rule	Action
Duplicate accident ids	Keep first, log the rest as rejected
Missing or unparseable date/time	Reject
Null lat/long	Reject
Coordinates outside UK bounds (roughly lat 49 to 61, lon -9 to 2) or exactly 0,0	Reject

Rule	Action
Severity not in {1,2,3}	Reject
Speed limit outside plausible values	Reject or set null (document which)
Coded values equal to -1 or 'data missing'	Convert to null, keep the row
Orphan vehicle or casualty rows (no matching accident)	Reject
Negative or absurd counts	Reject

Standardize while cleaning: parse `dd/mm/yyyy` and `HH:MM` into one timestamp, convert codes to labels using the STATS19 lookup guide, use consistent snake_case column names, and strip whitespace from text fields.

Also write a **data-quality summary** per run (rows in, rows passed, rows rejected per rule) into `ops.dq_results`. It makes a great report table and dashboard indicator.

8. Joining weather (the key design decision)

Do **not** call the API once per accident. There are 100k+ accidents per year, and you would hit rate limits.

Instead:

1. Assign each accident to a **grid cell** by rounding coordinates (for example 0.25 degrees, about 25 km). Store `grid_cell_id` in `dim_location`.
2. Take the distinct cell centres (a few hundred for the UK).
3. For each cell centre, request the **hourly historical weather** for the whole date range in one call (the API accepts a start and end date, and several locations per request). Useful variables: temperature, precipitation or rain, wind speed, cloud cover, weather code.
4. Store the result as `staging.weather_hourly` (cell id, timestamp, variables).
5. Join: accident timestamp **floored to the hour** + accident's grid cell = weather row.
6. Report how many accidents had no weather match, and log them.

This is worth a paragraph in the report: it explains the trade-off between spatial accuracy and API cost.

9. Aggregates and the model-ready table

Build these in `mart`:

- Accidents by **hour** and by **weekday** (and the hour x weekday heatmap)
- Accidents by **road type** and **weather condition**
- Severity share by speed limit, light condition, road surface
- Hotspots: counts per grid cell or local authority
- Vehicle-type and casualty-age summaries

Build one wide **`mart.model_ready`** table with one row per accident: the target (`severity`) plus features known **at the time of the accident** (hour, weekday, month, road type, speed limit, junction, light, surface, weather values, urban/rural, location cell, number of vehicles). Never edit it by hand; it is generated only by the pipeline.

10. Airflow DAG

One DAG, with one task per stage, so the graph tells the story:

```
# dags/accident_pipeline_dag.py (skeleton)
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import datetime

with DAG(
    dag_id='accident_pipeline',
    start_date=datetime(2026, 1, 1),
    schedule='@weekly',   # or @monthly: needs to be repeatable/scheduled
    catchup=False,
    default_args={'retries': 2},
) as dag:
    extract_accidents = PythonOperator(task_id='extract_accidents',
python_callable=...)
    extract_weather   = PythonOperator(task_id='extract_weather',
python_callable=...)
    load_staging      = PythonOperator(task_id='load_staging',
python_callable=...)
    clean_validate    = PythonOperator(task_id='clean_validate',
python_callable=...)
    join_weather      = PythonOperator(task_id='join_weather',
python_callable=...)
```

```
    build_mart        = PythonOperator(task_id='build_mart',
python_callable=...)
    dq_report         = PythonOperator(task_id='dq_report',
python_callable=...)

    [extract_accidents, extract_weather] >> load_staging >> clean_validate \
        >> join_weather >> build_mart >> dq_report
```

Tips:

- Put real logic in `src/` and keep the DAG thin; the DAG only calls functions.
- Make every task **idempotent** (truncate-and-reload a layer, or upsert) so retries don't duplicate data.
- Read DB credentials from environment variables, never from the DAG file.
- For the demo, screenshot the Airflow graph view with all tasks green, plus the logs.

11. Dashboard (Streamlit)

Needs **at least five meaningful views**, all reading from the `mart` schema:

1. Severity distribution (bar or donut)
2. Accidents by hour and weekday (heatmap or grouped bars)
3. Weather and road-condition impact (severity share by weather, surface, light)
4. Location hotspot map (`st.map`, pydeck, or folium)
5. Vehicle and casualty analysis (vehicle types, casualty age/sex)

Add sidebar filters (year, severity, road type, weather) and a one-line takeaway under each chart. The assignment grades **interpretation**, not just charts. Also add a small pipeline-health panel (last run time, rows loaded, rows rejected). Leave a page placeholder for the Part 2 prediction form.

12. Part 1 submission checklist

- [ ] Source code and ETL scripts
- [ ] Airflow DAG
- [ ] Database schema and sample populated tables (SQL file plus screenshots or CSV samples)
- [ ] Dataset source information and access instructions

- [ ] Architecture diagram covering source, ingestion, raw/staging, transformation, storage, analytics, and (later) MLOps layers
- [ ] Data dictionary and validation rules
- [ ] Streamlit app (or Tableau link)
- [ ] Report of about 8-10 pages
- [ ] Screenshots or execution evidence (successful DAG run, row counts, rejected log)
- [ ] README with setup and run instructions

PART 2: MLOps Extension

13. Problem framing and features

Task: multiclass classification of accident severity (Fatal / Serious / Slight). Fatal is rare (typically around 1% of records), so the problem is heavily imbalanced.

Pick features from `mart.model_ready`:

- Time: hour, weekday, month, is_weekend, is_rush_hour
- Road: road type, speed limit, junction detail, surface, urban/rural
- Environment: light condition, weather code, temperature, precipitation, wind
- Location: grid cell id or coarse lat/long, local authority
- Context: number of vehicles

Watch out for leakage. The number of casualties and any casualty-level fields are only known after the accident, and severity is defined from casualty outcomes. Exclude them from model inputs. The same applies to anything your API caller couldn't know at prediction time.

Put all feature engineering in `src/ml/features.py` so training and inference use identical code.

14. Train, validation and test split

Use a **chronological** split: train on the earliest period, validate on the next, test on the most recent. For example, with three years of data, train on years 1 and 2 (or the first 70% by date), validate on the next block, and test on the last. This mimics real deployment and avoids future data leaking into training. Document the exact date boundaries in the report.

15. Class imbalance

Try at least one, and compare against the plain baseline:

- **Class weights** (`class_weight='balanced'` for Random Forest, `sample_weight` for XGBoost/LightGBM)
- **Sampling** (SMOTE or random undersampling), applied to the **training set only**, never validation or test

Tune with the goal of **recall on Fatal and Serious**, since missing a severe accident is the costly error.

16. Model development and evaluation

- Baseline first (majority class or logistic regression), then Random Forest, XGBoost or LightGBM.
- Do light hyperparameter tuning on the validation set (grid or random search, a handful of runs is fine).
- Do **not** report accuracy alone; it looks great (about 80%+) just by predicting 'Slight' every time.
- Report on the test set: **per-class recall, precision, F1, macro-F1**, and a **confusion matrix**. Highlight false negatives for the Fatal and Serious classes.
- Add feature importances or SHAP for the interpretation section.

17. MLflow tracking and registry

For every training run, log:

- **Parameters:** model type, hyperparameters, imbalance method, split dates, feature list version
- **Metrics:** macro-F1, recall per class, precision per class, validation and test scores
- **Artifacts:** confusion matrix image, classification report, feature importance plot, the fitted pipeline
- **Tags:** data version or extraction date, git commit

Then:

1. Compare runs in the MLflow UI and pick a winner with a stated criterion (for example, best macro-F1 subject to a minimum Fatal recall).

2. **Register** the chosen model in the MLflow Model Registry.
3. Version it, and use stages or aliases (for example `staging` and `production`) to show the lifecycle.

Run MLflow as a service in Docker Compose with Postgres as the backend store, so the screenshots look professional and runs persist.

18. One inference pipeline

Package preprocessing and the model as **a single scikit-learn `Pipeline`** (a `ColumnTransformer` for encoding and scaling, then the classifier), and log that one object to MLflow. The API then receives raw field values and the pipeline does everything. This prevents training and serving skew, which is a central MLOps point and will be asked about in the viva.

19. FastAPI service

Endpoints to build:

- `POST /predict`: takes accident details, returns predicted severity plus class probabilities
- `GET /health`: liveness check
- `GET /model-info`: model name, version, training date, metrics
- `GET /metrics` (optional): Prometheus-style counters for latency and errors

```
# api/main.py (skeleton)
from fastapi import FastAPI
from pydantic import BaseModel
import mlflow.sklearn, time, logging

app = FastAPI()
model = mlflow.sklearn.load_model('models:/accident_severity/Production')

class AccidentInput(BaseModel):
    hour: int
    weekday: int
    month: int
    road_type: str
    speed_limit: int
    light_condition: str
```

```
    weather_condition: str
    temperature: float
    precipitation: float
    wind_speed: float
    num_vehicles: int
    latitude: float
    longitude: float

@app.post('/predict')
def predict(x: AccidentInput):
    start = time.time()
    # build a one-row DataFrame, call model.predict_proba, return label +
probs
    # log the input, prediction, and latency for monitoring
    ...
```

Validate inputs with Pydantic (ranges, allowed categories), log every request and prediction (needed for monitoring), and return clear errors rather than stack traces.

20. Docker

- `api/Dockerfile`: slim Python base image, install pinned requirements, copy the app, run `uvicorn`.
- `docker-compose.yml`: services for `postgres`, `airflow`, `mlflow`, `api`, `streamlit`, with credentials from `.env`.
- Test the full flow with `docker compose up` on a clean machine or clean clone; the README commands must work as written.

21. Integrating predictions into the dashboard

Add a Streamlit page 'Predict severity': input widgets for the same fields as the API, a button that calls `POST /predict`, and a display of the predicted class and probabilities. Optionally add a 'model health' page showing the monitoring charts from the next section.

22. Monitoring

Store every prediction request (inputs, output, timestamp, latency, error flag) in a Postgres table such as `ops.prediction_log`. Then compute, on a schedule (an Airflow DAG works well):

What	How
Input data quality	Share of nulls, out-of-range values, unseen categories per feature
Class drift	Compare the predicted (and, later, actual) severity distribution against the training distribution (PSI or a chi-square test)
Feature and location drift	PSI or KS test per numeric feature; compare the distribution of grid cells or lat/long against training data
Model performance	When true outcomes arrive (new STATS19 releases), recompute macro-F1 and per-class recall
False-negative rate	Share of actually Fatal or Serious accidents predicted as Slight
Latency and failures	p50/p95 latency, error rate, request counts

If you don't have live traffic, **simulate it**: replay the held-out test set, or a later year of data, through the API, and deliberately include a shifted batch (for example only winter, only one region) to demonstrate that drift detection fires. Evidently AI is a convenient open-source library for drift reports, or you can code PSI yourself.

23. Retraining criteria and model lifecycle

Define explicit, written triggers, for example:

- PSI above 0.2 on key features or on the class distribution
- Fatal recall or macro-F1 drops below a set threshold on newly labelled data
- False-negative rate above an agreed limit
- A new annual STATS19 release becomes available (scheduled retrain)
- Sustained rise in input-quality failures

Then document the lifecycle: ingest new data (Part 1 DAG) -> rebuild model-ready table -> retrain -> log to MLflow -> compare with the current production model -> promote only if better -> deploy the new container -> monitor. A lifecycle diagram belongs in the report.

24. Part 2 submission checklist

- [ ] Reproducible feature-engineering and training code
- [ ] Chronological split, documented
- [ ] Imbalance handling with before/after comparison
- [ ] MLflow experiments with params, metrics (recall, macro-F1) and confusion matrices
- [ ] Registered and versioned model
- [ ] Single preprocessing + model inference pipeline
- [ ] FastAPI service with predict, health, and model-info endpoints
- [ ] Dockerfile and docker-compose that run the whole stack
- [ ] Streamlit (or Tableau) integration with predictions
- [ ] Monitoring: input quality, class drift, location drift, false-negative rate, latency, failures
- [ ] Retraining criteria and model lifecycle write-up
- [ ] Updated architecture diagram including the MLOps layer, final report, and README

---

25. Where the marks are

Component	Part 1	Part 2
Problem understanding and design	5	5
Data source and ingestion	10	5
ETL, validation, and storage	20	5
Visualization and interpretation	10	5
Documentation and demonstration	5	5
Model development and evaluation	-	10
MLflow, registry, deployment, Docker	-	10
Monitoring and retraining design	-	5

Biggest single items: **ETL, validation and storage** (20) in Part 1, and **model evaluation** plus **MLflow/deployment** (10 each) in Part 2. Don't skimp on the rejected-records log or on imbalance-aware evaluation.

26. Common pitfalls

- Calling the weather API per accident instead of per grid cell and hour.
- Editing the final tables by hand, which the brief forbids; everything must come from the pipeline.
- Hardcoded passwords or API keys (use `.env`, and commit only `.env.example`).
- Judging the model on accuracy; use macro-F1 and per-class recall.
- Random train/test split instead of chronological.
- Resampling before splitting, which leaks information.
- Using casualty counts as features (leakage).
- Preprocessing outside the saved pipeline, so the API behaves differently from training.
- Leaving documentation for the end; the report, data dictionary and diagrams carry marks in both parts.
- Committing large raw data files to git; add `data/` to `.gitignore` and document the download step.

27. Definition of done

You're finished when someone can clone your repo, copy `.env.example` to `.env`, run `docker compose up`, trigger the Airflow DAG, open the Streamlit dashboard, submit a prediction through the API, and see the drift and performance numbers, all by following your README with no manual database edits.